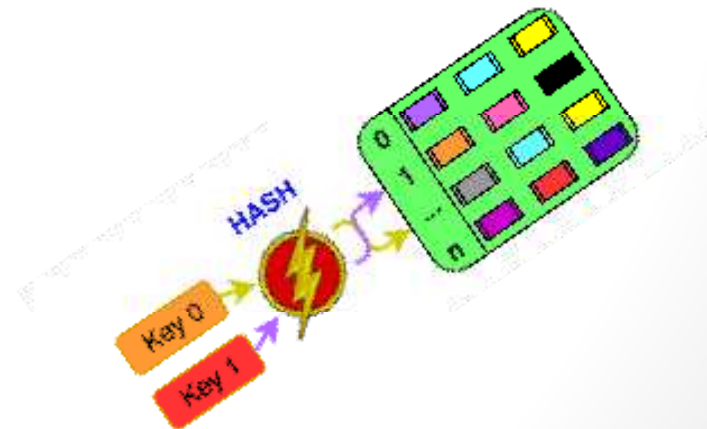
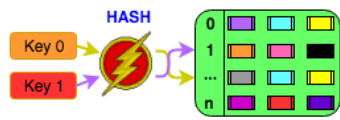

Estrutura de Dados

Tabela Hash

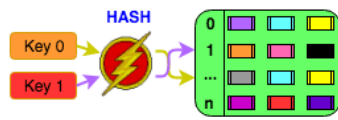
Profa. Dra. Lúcia Guimarães





- **Introdução – Tabela Hash**

- **Tabela Hash** tem como principal objetivo resolver o problema da necessidade de **encontrar ou armazenar** eficientemente um item em uma coleção.
- **Por Exemplo:**
 - Em uma lista de 10 mil palavras se quisermos verificar se uma determinada palavra está na lista, seria ineficiente comparar sucessivamente a palavra com todos os 10 mil itens até encontrarmos uma correspondência, mesmo que a lista estivesse ordenada



- **Introdução – Tabela Hash**

- **Hashing** é uma técnica para tornar as coisas mais eficientes, reduzindo efetivamente a busca desde o início.
- É uma estrutura de dados onde as posições de **inserção e busca** são calculadas através de uma função que visa distribuir os elementos aleatoriamente ao longo de um vetor.
- **A ideia central é**
 - Utilizar uma função, aplicada sobre parte da informação **(chave)**,
 - Para **retornar o índice onde a informação deve ou deveria estar armazenada.**

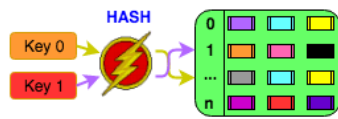
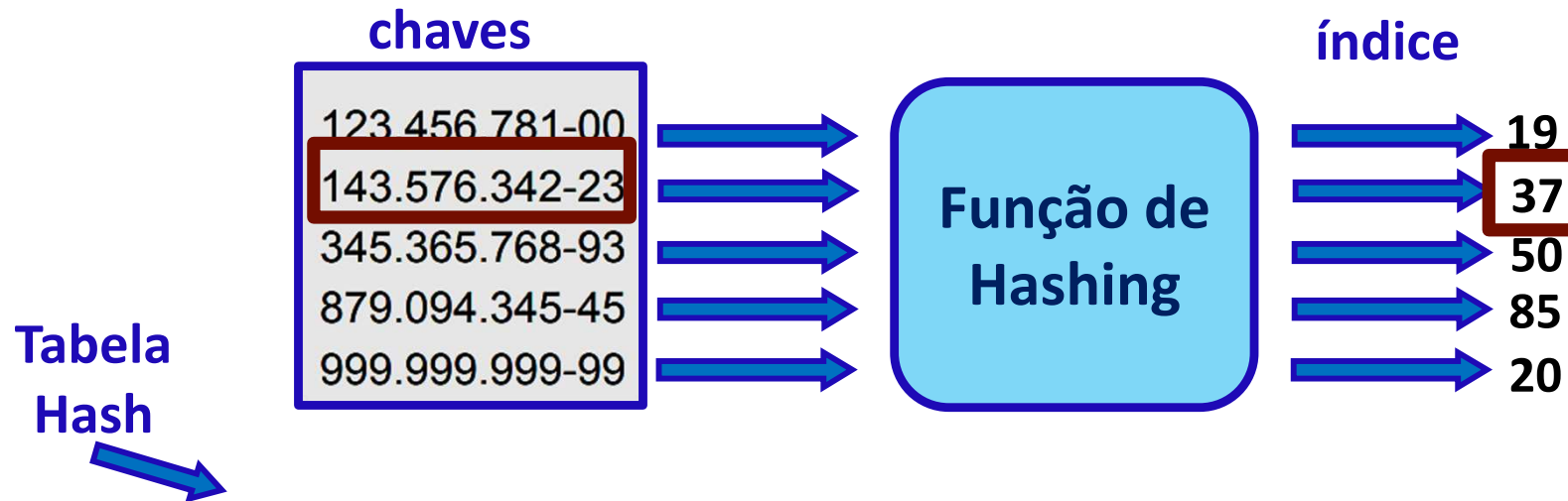
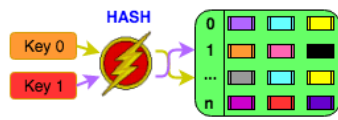


Tabela Hash

- EXEMPLO



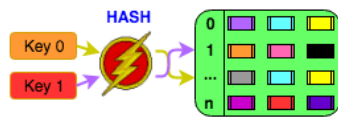
19	123.456.781-00; Fausto Silva; Av. Canal. Nº 45.
20	
...	
37	143.576.342-23; Carla Perez; Rua Celso Oliva. Nº 27.
...	
50	345.365.768-93; Gugu Liberato; Av. Atlântica. S/N.
...	
85	879.094.345-45 ; Hebe Camargo; Rua B. Nº 100.
...	



- **Definições:**

- **Tabela Hash**

- **Ou de dispersão, ou de espalhamento**
- Estrutura de Dados especial, que **associa chaves de pesquisa a valores**
- Seu objetivo é a partir de uma chave simples, fazer **uma busca rápida e obter o valor desejado**
- A ideia é **utilizar uma função**, aplicada sobre parte da informação (chave), para **retornar o índice onde a informação deve ou deveria estar armazenada.**



- **Definições:**

- **Função de Hashing ou Função de Espalhamento**

- Função que mapeia a chave para um índice de um arranjo (vetor)
- Ou seja, é a responsável por gerar um índice a partir de determinada chave
- Portanto, **transforma cada chave em um índice da tabela de hash.**

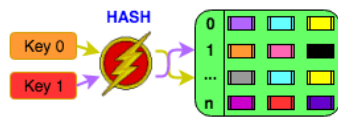


Tabela Hash

- Definições:

- Função de Hashing ou Função de Espalhamento

- Transforma cada chave em um índice da tabela de hash.

- Exemplo:

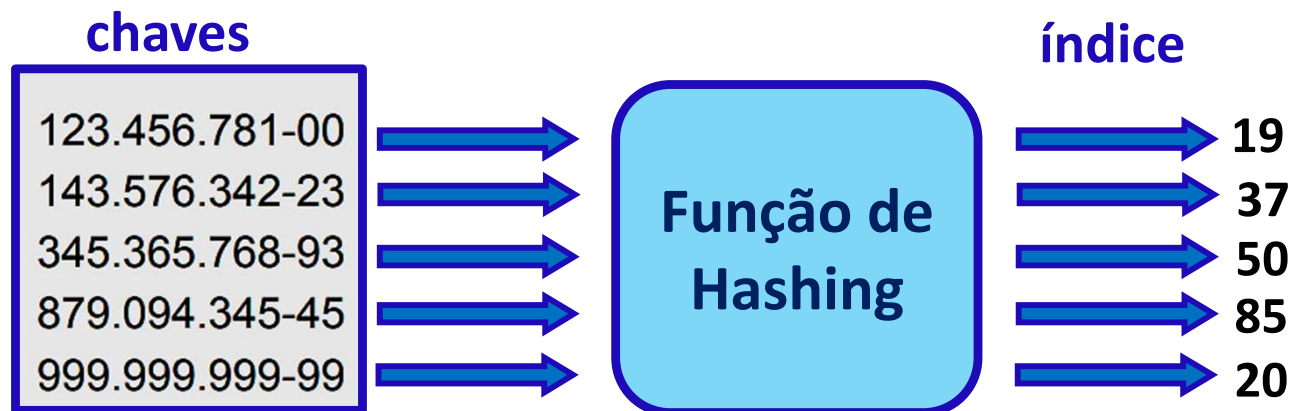
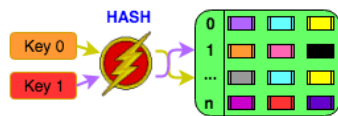


Tabela Hash



19	123.456.781-00; Fausto Silva; Av. Canal. Nº 45.
20	
...	
37	143.576.342-23; Carla Perez; Rua Celso Oliva. Nº 27.
...	
50	345.365.768-93; Gugu Liberato; Av. Atlântica. S/N.
...	
85	879.094.345-45 ; Hebe Camargo; Rua B. Nº 100.
...	



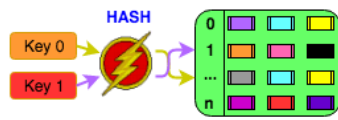
- **Introdução**

- **Exemplo**

- A posição de um elemento é obtida aplicando-se ao elemento a **função de hash** que devolve a sua posição na tabela.
 - Daí basta verificar se o elemento realmente está nesta posição.
 - O objetivo então é transformar a chave de busca em um índice na tabela

i	0	1	2	3	4	5	6	7	8	9
A[i]	-1	-1	-1	-1	34	45	-1	67	78	89

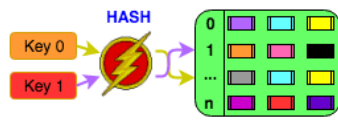
- Observe que:
 - -1 indica que não existe elemento naquela posição
 - **função de hash:** $x\%10$ (resto da divisão por 10).



- **Definições:**

- **Função de Hashing ou Função de Espalhamento**

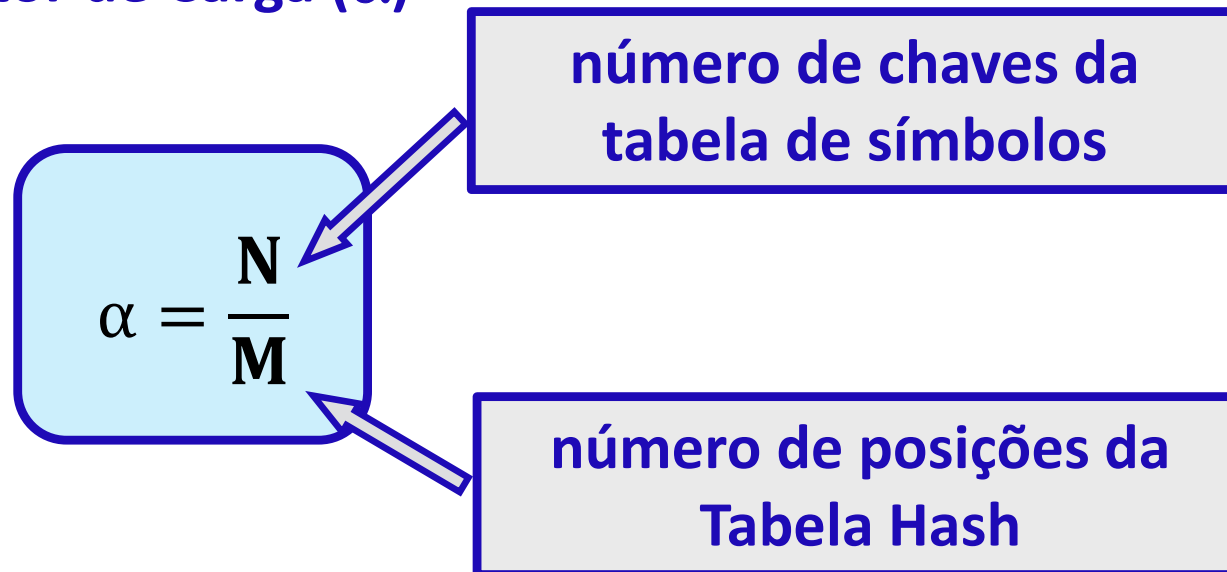
- Responde a pergunta: “*Em qual posição da Tabela de Hash devo colocar esta chave?*”
- Espalha as chaves pela Tabela de Hash
- Associa um valor hash a cada chave



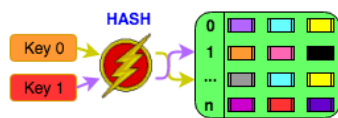
- **Definições:**

- **Parâmetro Importante**

- **Fator de Carga (α)**



- O fator de carga permite verificar a proporção de distribuição das chaves.
- Varia de 0.0 e 1.0



número de chaves da tabela de símbolos

$$\alpha = \frac{N}{M}$$

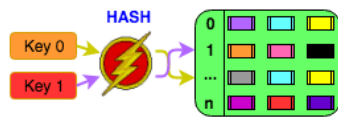
número de posições da Tabela Hash

- **Definições:**

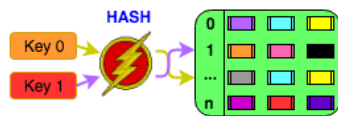
- **Parâmetros Importantes**

- **Fator de Carga (α)**

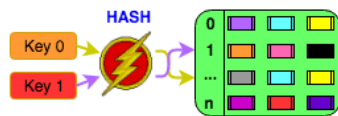
- Quanto maior o fator de carga , menos espaços disponíveis na tabela e maior a chance de colisão para as novas chaves
- É muito comum estabelecer um **limite superior** ao qual o fator de carga sempre deve estar abaixo
- Cada implementação de tabelas Hash pode escolher sua estratégia.
- Mas um exemplo seria ao chegar em um fator de carga igual a 0.8 se faz uma realocação da tabela de modo a aumentar o **valor de M**, ou seja, o número total de espaços disponíveis



- **Como representar Tabelas Hash???**
 - **Vetor : (Exemplo 1 – Introdução)**
 - Cada posição do vetor guarda uma informação.
 - Se a **função de hashing** aplicada a um conjunto de elementos,
 - determinar as informações $l_1, l_2, \dots, l_n$,
 - então o vetor $V[1... n]$ é usado para representar a tabela hash



- **Como representar Tabelas Hash???**
 - **Vetor + Lista Encadeada (Exemplo2 – Introdução)**
 - O vetor contém ponteiros para as listas que representam as informações.
- **Função de Hashing**
 - É a responsável por gerar um **índice** a partir de uma determinada **chave**.
 - O **ideal** é que a função forneça **índices únicos** para conjunto das chaves de entrada possíveis.
 - A função de Hashing é **extremamente importante**, pois ela é responsável por distribuir as informações pela Tabela Hash

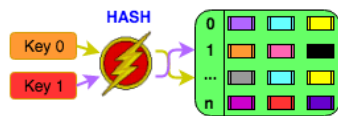


- **Tabelas Hash**

- Considerando o exemplo da função $x\%10$
- Suponha os elementos 23, 42, 33, 52, 12, 58
- Há mais de um elemento para certas posições.
- **Uma Solução:**
 - Aumentar a tabela para, por exemplo 17 posições e usar a função $x\%17$

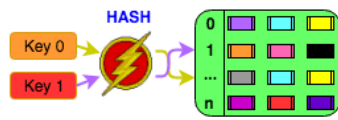
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
-1	52	-1	-1	-1	-1	23	58	42	-1	-1	-1	12	-1	-1	-1	33

- A função de hash pode ser escolhida à **vontade** de forma a atender da **melhor forma a distribuição**.

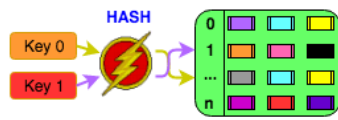


- **Tabelas Hash**

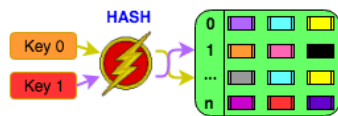
- A escolha da função é a parte **mais importante**.
- É sempre melhor escolher uma função que use uma tabela com um **número razoável de elementos**.
- A escolha da função é um compromisso entre a **eficiência** na busca o gasto de **memória**.
- A ideia central das técnicas de hash é sempre **espalhar** os elementos de forma que os mesmos sejam rapidamente encontrados.



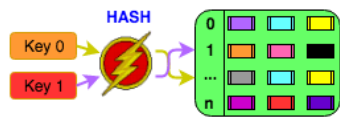
- **Função Hashing (ou função de Dispersão)**
 - Idealmente, uma função de dispersão deve satisfazer às seguintes condições:
 - produzir um número baixo de colisão;
 - ser facilmente computável;
 - ser uniforme.
 - Há muitas maneiras de determinar uma função de *hash*.
 - Método da Divisão
 - Método da Dobra
 - Método da Multiplicação



- **Função Hashing (ou função de Dispersão)**
 - **Método da Divisão**
 - Fácil e eficiente além de ser muito empregado
 - A operação “resto da divisão por” (módulo – % em C) é a maneira mais direta de transformar valores em índices.
 - **$h(x) = x \% M$ (o resto de x dividido por M)**
 - Uma função de **hash** precisa garantir que o valor retornado seja um índice válido para uma das células da tabela.

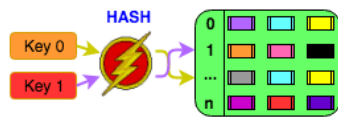


- **Função Hashing (ou função de Dispersão)**
 - **Método da Divisão**
 - $h(x) = x \% M$ (o resto de x dividido por M)
 - Se o conjunto é de inteiros e a tabela é de M elementos, a função de hash pode ser simplesmente $x \% M$.
 - Se o conjunto é de valores fracionários entre 0 e 1 com 8 dígitos significativos, a função de hash pode ser $\text{floor}(x * 10^8) \% M$.
 - Se são números entre s e t , a função pode ser $\text{floor}((x - s) / (t - s) * M)$
 - **A escolha é bem livre, mas o objetivo é sempre espalhar ao máximo dentro da tabela os valores da função para eliminar as colisões.**



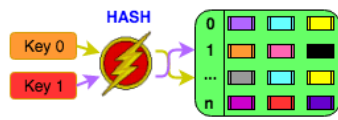
- **Colisões**

- Quando duas ou mais chaves geram o **mesmo endereço da Tabela Hash**
- É comum ocorrer colisões.
- Principais causas:
 - em geral o número N de chaves possíveis é **muito maior** que o número de **entradas** disponíveis na tabela.
 - **não** se pode garantir que as funções de hashing possuam um **bom potencial de distribuição (espalhamento)**.



- **Colisões**

- No caso geral, não há informações sobre os elementos e seus valores.
- É comum conhecer apenas a quantidade máxima de elementos que a tabela conterà.
- **Problema:** Como tratar os elementos cujo valor da função de hash é o mesmo? ??????



- **Colisões**

- Para tratar as colisões, pode-se:
 - colocar o elemento na primeira posição livre seguinte e
 - considerar a tabela como circular (o elemento seguinte ao último $a[n-1]$ é o primeiro $a[0]$).
 - Isso se aplica tanto na inserção de novos elementos quanto na busca.
- Esta técnica é conhecida como Lista Linear, Sondagem Linear
- Função Hash

$$h(x,k) = (h'(x) + k) \bmod M \quad 0 \leq k \leq M-1$$

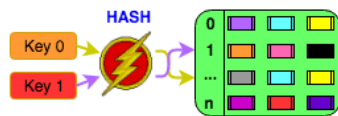


Tabela Hash

- Colisões
 - Lista Linear, Sondagem Linear – Exemplo

$$h(x,k) = (h'(x) + k) \bmod M \quad 0 \leq k \leq M-1$$

$$m=10 \quad 0 \leq k \leq 9$$

i	0	1	2	3	4	5	6	7	8	9
A[i]	-1	-1	42	23	33	-1	-1	-1	-1	-1

Valores: 42, 23, 33, 52, 12, 58

$$42 \Rightarrow h(42,0) = (42 + 0) \bmod 10 = 2$$

$$23 \Rightarrow h(23,0) = (23 + 0) \bmod 10 = 3$$

$$33 \Rightarrow h(33,0) = (33 + 0) \bmod 10 = 3 \quad \text{Ocupado!!!!!!}$$

$$33 \Rightarrow h(33,1) = (33 + 1) \bmod 10 = 4$$

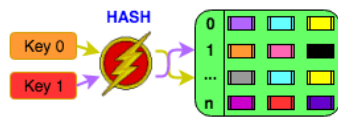


Tabela Hash

- Colisões
 - Lista Linear, Sondagem Linear – Exemplo

$$h(x,k) = (h'(x) + k) \bmod M \quad 0 \leq k \leq M-1$$

$$m=10 \quad 0 \leq k \leq 9$$

i	0	1	2	3	4	5	6	7	8	9
A[i]	-1	-1	42	23	33	52	-1	-1	-1	-1

Valores: 42, 23, 33, 52, 12, 58

$$52 \Rightarrow h(52,0) = (52 + 0) \bmod 10 = 2 \text{ Ocupado!!!!!}$$

$$52 \Rightarrow h(52,1) = (52 + 1) \bmod 10 = 3 \text{ Ocupado!!!!!}$$

$$52 \Rightarrow h(52,2) = (52 + 2) \bmod 10 = 4 \text{ Ocupado!!!!!}$$

$$52 \Rightarrow h(52,3) = (52 + 3) \bmod 10 = 5$$

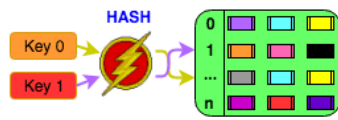


Tabela Hash

- Colisões
 - Lista Linear, Sondagem Linear – Exemplo

$$h(x,k) = (h'(x) + k) \bmod M \quad 0 \leq k \leq M-1$$

$$m=10 \quad 0 \leq k \leq 9$$

i	0	1	2	3	4	5	6	7	8	9
A[i]	-1	-1	42	23	33	52	12	-1	58	-1

Valores: 42, 23, 33, 52, 12, 58

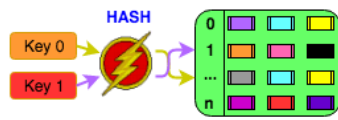
$$12 \Rightarrow h(12,0) = (12 + 0) \bmod 10 = 2 \text{ Ocupado!!!!!}$$

$$12 \Rightarrow h(12,1) = (12 + 1) \bmod 10 = 3 \text{ Ocupado!!!!!}$$

$$12 \Rightarrow h(12,2) = (12 + 2) \bmod 10 = 4 \text{ Ocupado!!!!!}$$

$$12 \Rightarrow h(12,3) = (12 + 3) \bmod 10 = 5 \text{ Ocupado!!!!!}$$

$$12 \Rightarrow h(12,4) = (12 + 4) \bmod 10 = 6$$



- **Colisões**

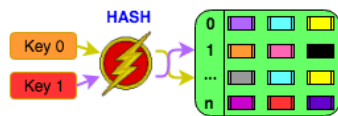
- Lista Linear, Sondagem Linear – **Exemplo**

$$h(x,k) = (h'(x) + k) \bmod M \quad 0 \leq k \leq M-1$$

i	0	1	2	3	4	5	6	7	8	9
A[i]	-1	-1	42	23	33	52	12	-1	58	-1

Desvantagem:

- a tendência de formação de **grupos de posições ocupadas consecutivas**,
- fazendo com que a primeira posição vazia, na prática, possa ficar muito longe da posição original, dada pela função de hash



- **Colisões**

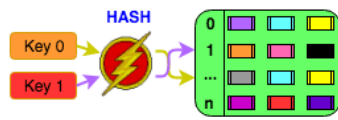
- Lista Linear, Sondagem Linear – **Exemplo**

$$h(x,k) = (h'(x) + k) \bmod M \quad 0 \leq k \leq M-1$$

i	0	1	2	3	4	5	6	7	8	9
A[i]	-1	-1	42	23	33	52	12	-1	58	-1

Desvantagem:

- Dificulta a remoção de dados
- Aumenta a complexidade para a busca nos casos de colisão

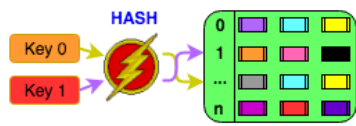


• EXERCÍCIOS

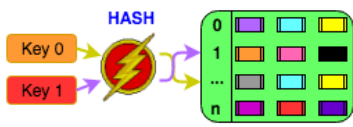
1. Utilizando a função hash **de divisão com sondagem linear**, com a regra de colisão de sondagem linear. Demonstre a inserção das chaves: 5, 28, 19, 15, 20, 33, 12, 7 e 10 considerando:

- a) $M=9$
- b) $M=11$
- c) $M=15$

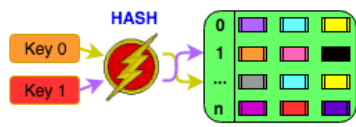
$$h(x,k) = (h'(x) + k) \bmod M \quad 0 \leq k \leq M-1$$



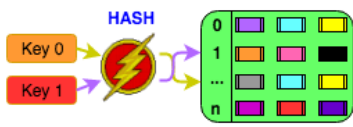
- **Colisões – Duplo hashing ou rehash**
 - Se a tabela está muito cheia a busca sequencial pode levar a um número muito grande de comparações.
 - No pior caso (N elementos ocupados), deve-se percorrer os N elementos antes de encontrar o elemento ou concluir que ele não está na tabela.
 - Uma forma de permitir um espalhamento maior é fazer com que o deslocamento em vez de 1 seja dado por uma segunda função de *hash*.



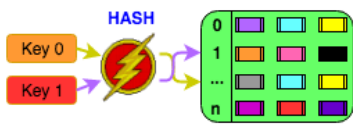
- **Colisões – Duplo hashing ou rehash**
 - Essa segunda função de *hash* tem que ser escolhida com cuidado, não deve gerar um valor nulo (*loop* infinito).
 - Deve ser tal que a soma do índice atual com o deslocamento (módulo M) dê sempre um número diferente até que os N números sejam verificados.
 - Para isso M e o valor desta função devem ser primos entre si.



- **Colisões – Duplo hashing ou rehash**
 - Uma maneira é escolher M primo e garantir que a segunda função de *hash* tenha um valor K menor que M . Dessa forma M e K são primos entre si
 - Existem duas funções de *Hash*:
 - Uma para usar normalmente e
 - Outra para usar quando há colisões.



- **Colisões – Duplo hashing ou rehash**
 - Existem duas funções de *Hash*:
 - $h1(x) = (x \% M) = C1$
 - $h2(x) = (x \% (M-1)) + 1 = C2 \rightarrow$ usada quando há colisões
 - Para calcular o primeiro índice usa-se $C1$;
 - Para calcular o segundo índice (se existir colisões) usa-se $(C1 + C2) \% M$;
 - Para calcular o terceiro índice (se também houver colisões) usa-se $(C1 + 2C2) \% M$, depois $(C1 + 3C2) \% M$, etc.



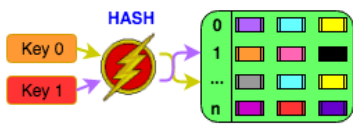
- **Colisões – Duplo hashing ou rehash – Exemplo**

0	1	2	3	4	5	6
-1	-1	-1	-1	-1	-1	-1

- Inserir os elementos 5, 10, 12, e 19 na tabela com $M=7$
- $h1(x) = x \% 7 = \mathbf{C1}$
- $h2(x) = (x \% 6) + 1 = \mathbf{C2}$

$$h1(x) = (x \% M) = C1$$

$$h2(x) = (x \% (M-1)) + 1 = C2$$



- Colisões – Duplo hashing ou rehash – Exemplo

0	1	2	3	4	5	6
-1	-1	-1	10	-1	5	12

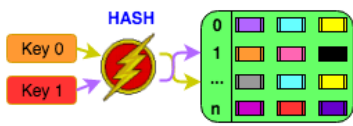
- Inserir os elementos 5, 10, 12, e 19 na tabela com $M=7$
- $h1(x) = x \% 7 = C1$
- $h2(x) = (x \% 6) + 1 = C2$

5 $\rightarrow h1(5) = 5 \rightarrow$ Posição 5 está vazia

10 $\rightarrow h1(10) = 3 \rightarrow$ Posição 3 está vazia

12 $\rightarrow h1(12) = 5 \rightarrow$ Posição 5 está ocupada, faz-se $h2$

12 $\rightarrow h2(12) = 1 \rightarrow (C1+C2)\%7=6$ Posição 6 está vazia



• Colisões – Duplo hashing ou rehash – Exemplo

0	1	2	3	4	5	6
19	-1	-1	10	-1	5	12

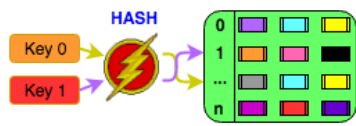
- Inserir os elementos 5, 10, 12, e 19 na tabela com $M=7$
- $h1(x) = x \% 7 = C1$
- $h2(x) = (x \% 6) + 1 = C2$

• **E agora o 19??? Faça você!!!!!!**

19 $\rightarrow h1(19) = 5 \rightarrow$ Posição 5 está ocupada, faz-se $h2$

19 $\rightarrow h2(19) = 2 \rightarrow (C1+C2)\%7=0$ Posição 0 está vazia

$$\left. \begin{array}{l} C1 = 19 \% 7 = 5 \\ C2 = (19 \% 6) + 1 = 2 \end{array} \right\} C1 + C2 = 7$$



- **Colisões – Duplo hashing ou rehash**
 - Como a seleção da segunda função de *hash* é livre pode se escolher um valor fixo.
 - **Exemplo** : Inserir os elementos 25, 37, 48, 59, 32, 44, 70, 81 (nesta ordem) com $M=11$.

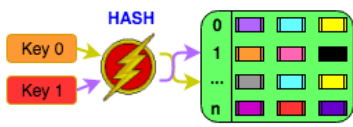
$$h1(x) = x \% 11 = C1$$

FAÇA VOCÊ !!!!

$$h2(x) = 3 = C2$$

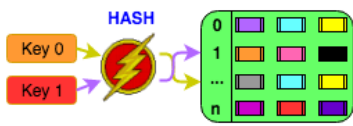
Para calcular o segundo índice (se existir colisões) usa-se $(C1+C2) \% N$;

Para calcular o terceiro índice (se também houver colisões) usa-se $(C1+2C2) \% M$, depois $(C1+3C2) \% M$, etc.



- **Comentários:**

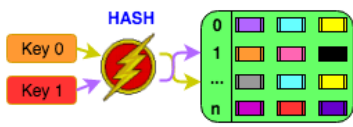
- A probabilidade de colisão pode ser reduzida usando uma tabela suficientemente grande em relação ao número total de posições a serem ocupadas.
 - Por exemplo, uma tabela com 1000 entradas para uma empresa que deseja armazenar 500 posições haveria uma probabilidade de 50% de colisão, se fosse feita a inserção de uma nova chave.
- Considera-se uma tabela *hash* bem dimensionada,
 - Média 1,5 acessos à tabela para encontrar um elemento.



- **Comentários:**

- Hash Perfeita

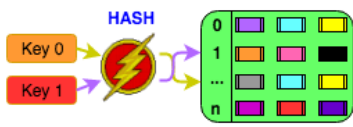
- O ideal para a função *hash* é que sejam sempre fornecidos índices únicos para as chaves de entrada.
 - A função perfeita (*hash* perfeita) seria:
 - para quaisquer entradas A e B, sendo A diferente de B, fornecesse saídas diferentes.



- **Comentários:**

- Hash Perfeita

- A tabela deve conter o mesmo número de elementos.
 - Nem sempre o número de elementos é conhecido a priori.
 - Na prática, funções *hash* perfeitas ou quase perfeitas são encontradas apenas onde a colisão é intolerável
 - por exemplo, nas funções *hash* da criptografia, ou quando se conhece previamente o conteúdo da tabela armazenada.



• EXERCÍCIOS

1. Utilizando a função hash: C1: $h(k) = k \% M$, com a regra de colisão: $h2(x) = (x \% (M-1) + 1) \% M = C2$. Para calcular o primeiro índice usa-se C1; Para calcular o segundo índice (se existir colisões) usa-se $(C1+C2) \% M$; Para calcular o terceiro índice (se também houver colisões) usa-se $(C1+2C2) \% M$, depois $(C1+3C2) \% M$. Demonstre a inserção das chaves: 5, 28, 19, 15, 20, 33, 12, 7 e 10 considerando:

a) $M = 9$

b) $M = 11$

5, 28, 19, 15, 20, 33, 12, 7 e 10

N=9

0	1	2	3	4	5	6	7	8
19	28	20	12		5	15	7	33

$$h1(x) = (x \% M) = C1$$

$$h2(x) = (x\%(M-1)) + 1 = C2$$

$$(C1 + C2)\%M$$

$$(C1 + 2C2)\%M\ldots\ldots$$

$$5 \rightarrow h1(5) = 5\%9 = 5$$

$$28 \rightarrow h1(28) = 28\%9 = 1$$

$$19 \rightarrow h1(19) = 19\%9 = 1$$

$$h2(19) = 19\%8 + 1 = 4$$

$$(1 + 4)\%9 = 5$$

$$(1 + 2*4)\%9 = 0$$

$$15 \rightarrow h1(15) = 15\%9 = 6$$

$$20 \rightarrow h1(20) = 20\%9 = 2$$

$$33 \rightarrow h1(33) = 33\%9 = 6$$

$$h2(33) = 33\%8 + 1 = 2$$

$$(2 + 6)\%9 = 8$$

$$12 \rightarrow h1(12) = 12\%9 = 3$$

$$7 \rightarrow h1(7) = 7\%9 = 7$$

5, 28, 19, 15, 20, 33, 12, 7 e 10

N=9

0	1	2	3	4	5	6	7	8
19	28	20	12	10	5	15	7	33

$$h1(x) = (x \% M) = C1$$

$$h2(x) = (x \% (M-1)) + 1 = C2$$

$$(C1 + C2) \% M$$

$$(C1 + 2C2) \% M \dots\dots\dots$$

$$10 \rightarrow h1(10) = 10 \% 9 = 1$$

$$h2(10) = 10 \% 8 + 1 = 3$$

$$(1 + 3) \% 9 = 4$$

$$h1(x) = (x \% M) = C1$$

$$h2(x) = (x\%(M-1)) + 1 = C2$$

$$(C1 + C2)\%M$$

$$(C1 + 2C2)\%M\ldots\ldots$$

5, 28, 19, 15, 20, 33, 12, 7 e 10

N=11

0	1	2	3	4	5	6	7	8	9	10
33	12			15	5	28	7	19	20	10

$$5 \rightarrow h1(11) = 5\%11 = 5$$

$$28 \rightarrow h1(28) = 28\%11 = 6$$

$$19 \rightarrow h1(19) = 19\%11 = 8$$

$$15 \rightarrow h1(15) = 15\%11 = 4$$

$$20 \rightarrow h1(20) = 20\%11 = 9$$

$$33 \rightarrow h1(33) = 33\%11 = 0$$

$$12 \rightarrow h1(12) = 12\%11 = 1$$

$$7 \rightarrow h1(7) = 7\%11 = 7$$

$$10 \rightarrow h1(10) = 10\%11 = 10$$

- **EXERCÍCIOS**

1. Utilizando a função hash: C1: $h(k) = k \% M$, com a regra de colisão: $h2(x) = (x \% (M-1)) + 1 = C2$. Para calcular o primeiro índice usa-se C1; Para calcular o segundo índice (se existir colisões) usa-se $(C1+C2) \% N$; Para calcular o terceiro índice (se também houver colisões) usa-se $(C1+2C2) \% M$, depois $(C1+3C2) \% M$. Demonstre a inserção das chaves: 5, 28, 19, 15, 20, 33, 12, 7 e 10 considerando:

a) $M=9$

b) $M = 11$